Problem K. 密钥恢复

你知道吗：在选拔 *ISIJ* 选手的过程中，*ISTJ* 和 *ISFJ* 并不会获得额外加分。

—拌鸡块君

这是一道交互题。

Kevin 不满意自己的 MBTI，因此他找哆啦 A 梦要了一台神奇的机器：MBTI 转换器。

MBTI 转换器一次可以对 8 个人的 MBTI 属性进行转换。它会基于这些人的初始 MBTI 和一个 32 比特的密钥 k ，为每个人生成一个 “随机” 的新 MBTI。

Kevin 认真研究了这台机器，发现其生成的 “随机”MBTI 实际上遵循一定的模式。该机器主要包含三种打乱 MBTI 的基本结构：“`xor`”, “`perm`” 和 “`mix`” 。每种结构均接受 8 个人的 MBTI 作为输入，并会输出 8 个 MBTI 作为结果。在每个结构中，输入的 8 个 MBTI 依次记为 $in_0, in_1, in_2, \cdots, in_7$ ，输出的 8 个 MBTI 依次记为 $out_0, out_1, out_2, \cdots, out_7$ 。为了方便，Kevin 根据下表将每种 MBTI 用一个十六进制数代替。

INFP	INFJ	INTP	INTJ	ISFP	ISFJ	ISTP	ISTJ
0	1	2	3	4	5	6	7
ENFP	ENFJ	ENTP	ENTJ	ESFP	ESFJ	ESTP	ESTJ
8	9	a	b	c	d	e	f

“`xor`” 结构：

`xor` 结构根据密钥 k 对 MBTI 的特定属性进行翻转。

令 k_j 表示密钥 k 的第 j 个二进制位 $(0 \le j < 32)$，则 `xor` 结构的功能可以表示为：

$$out_i = in_i \oplus (\overline{k_{4i+3}k_{4i+2}k_{4i+1}k_{4i}})_2$$

其中，$\oplus$ 表示按位异或 [1]。

“`perm`” 结构：

`perm` 结构对 MBTI 进行替换，每种 MBTI 会被替换为另一个固定的 MBTI。

`perm` 结构的功能可以表示为：

$$out_i = p_{in_i}$$

其中，p 是一个固定的长度为 16 的数组，值如下表所示：

p_0	p_1	p_2	p_3	p_4	p_5	p_6	p_7	p_8	p_9	p_{10}	p_{11}	p_{12}	p_{13}	p_{14}	p_{15}
7	0	5	6	b	f	4	3	1	d	c	e	8	9	a	2

“`mix`” 结构：

`mix` 结构用于混合不同人的 MBTI，每个输出由两个输入决定。

`mix` 结构需要用到一个长度为 8 的中间数组 t ，其定义为：

$$t_i = \begin{cases} in_i \ll 1 & 0 \le in_i < 8 \\ ((in_i \oplus 8) \ll 1) \oplus 3 & 8 \le in_i < 16 \end{cases}$$

其中，$\oplus$ 表示按位异或 [1] ，$\ll$ 表示四位二进制数的逻辑左移 [2] 。

`mix` 结构的功能可以表示为：

$$out_i = in_{i \ggg 1} \oplus (t_{(i \ggg 1) \oplus 1})$$

其中，$\oplus$ 表示按位异或 [1]，$\ggg$ 表示三位二进制数的循环右移 [3] 。

初始的 8 个 MBTI，将依次经过 19 个结构，最后得到输出的 8 个 MBTI。这些结构依次是：`xor, perm, mix, xor, perm, mix, xor, perm, mix, xor, perm, mix, xor, perm, mix, xor, perm, mix, xor` ，即连续六轮的 “`xor, perm, mix`” 再加一次的 “`xor`” 。

使用这台机器的人通常都希望能获得某个特定的 MBTI。可惜的是，哆啦 A 梦在制造这台机器时将密钥 k 封装在了机器内部，且拒绝告诉 Kevin 密钥的值。于是，Kevin 找到了聪明的你。Kevin 希望你构造一些输入（不超过 4096 个，否则机器可能会被损坏），测试运行得到对应的输出，然后从中分析出密钥的值。这样他就可以写一个程序预测每种输入对应的输出，使大家都能得到自己想要的 MBTI。

你能完成这个任务吗?

[1] 按位异或是指将两个二进制数字的每一位进行模 2 意义下的加法运算。例如：$(0011)_2 \oplus (0101)_2 = (0110)_2$ 。

[2] $x \ll y$ 表示将二进制数 x 逻辑左移 y 位，溢出部分舍弃，空出部分补 0。例如：$(1100)_2 \ll 1 = (1000)_2, (1001)_2 \ll 1 = (0010)_2$ 。

[3] $x \ggg y$ 表示将二进制数 x 循环右移 y 位。例如：$(101)_2 \ggg 1 = (110)_2, (110)_2 \ggg 1 = (011)_2$ 。

Interaction Protocol

要运行 MBTI 转换器，请输出 “`?` $in_7in_6in_5in_4in_3in_2in_1in_0$” ($in_i \in \{0-9, a-f\}$)，其中 in_i 是第 i 个输入 MBTI 对应的十六进制值。你可以进行这样的操作至多 4096 次。在清空输出缓冲区后，请以 “$out_7out_6out_5out_4out_3out_2out_1out_0$” ($out_i \in \{0-9, a-f\}$) 的格式读入 MBTI 转换器的运行结果。

要回答 k 的值，请输出 “`!` k” ($0 \le k < 2^{32}$) ，其中 k 是你认为的密钥 k 的值的**十进制**表示。你可以进行这样的操作至多一次。在操作后，你应该立即退出你的程序。

如果你运行 MBTI 转换器的次数超过了 4096 次，或是你的输出中包含了非法的字符，交互器将会输出 “`-1`” 。在收到这样的回复后，你应该立即退出你的程序，此时你会获得 “`Wrong Answer`” 的评测结果。否则，评测结果是不确定的，因为你将尝试从一个已经关闭的输出流中读取数据。

清空输出缓冲区可以使用以下方式：

- C 和 C++ 使用 `fflush(stdout)` （如果您使用 `printf`）或 `cout.flush()` （如果您使用 `cout`）。
- Java 和 Kotlin 使用 `System.out.flush()` 。
- Python 使用 `stdout.flush()` 。

Example

standard input	standard output
acf7e10b 105092e0	? 04f37255 ? 9090cfca ! 998244353

Note

以下信息可能对解题有所帮助：

1. 输入和输出的 in_i, out_i 之间没有空格，请不要输出多余的空格。输出的内容**不是**十六进制数，请**不要**忽略开头可能存在的 0 。
2. k 的范围是 $[0, 2^{32}-1]$ ，在输出答案时，请注意避免输出负数。
3. 本题的下发文件中包含了 “interact.hpp”，你可能需要该文件辅助进行本地调试。有关该文件的用法，你可以参考下发文件中的 “local_test_example.cpp” 。
4. 裁判程序**不是自适应的**。也就是说，交互器中的 k 在交互开始前就已经确定，不会随着你的交互而改变。
5. 本题共有 64 个测试点。